

V2



Thought for 2m 7s >

結論から言うと：

- いま話してきた 拡張 V2 要求仕様の一部は、現行 **Decision Console / Diagnostics / Portfolio** に“指標追加・チャート追加”の形で統合可能 です。
- ただし、
 - マルチアーム実験
 - オンライン最適化 (バンディット系)
 - 長期 LTV / フォーキャスト最適化
 - ガバナンス (フェアネス・安全装置)みたいな「意思決定の単位そのものが違うもの」は、既存ページに押し込むと UI/UX が崩壊するので、**V2** として別モジュール (別ページ) に切り出した方が合理的です。

なのでここでは、

V2 (というより、もうこれ以上はできないレベル) の Claude 向け仕様として、**既存 v1 を前提に“上に載せる”拡張 V2 全体仕様**をまとめます。
Claude に丸投げできるよう、モジュール構成・API・UI・Python/Wolfram コードの骨格まで書きます。

CQOx V2 - Advanced Decision & Optimization Suite (Claude向け仕様)

0. 全体コンセプト

0.1 ゴール

- v1:**
 - 単一施策ごとの **因果推論 ($\Delta\mathbb{Y}$, CAS, リスク)** と
 - シンプルな **ポートフォリオ最適化 (静的 Pareto)** を提供
- V2:** その上に以下を追加する：
 - Multi-Arm Causal Design Studio**
 - A/B/n, 多段階施策 (割引率 0/5/10/20% など)、連続 dose-response を扱える。
 - Experiment Orchestrator & Bandit Control Tower**
 - オンラインで配信比率を更新する バンディット / 強化学習ベースの運用レイヤ。
 - Growth & LTV Planning Studio**
 - $\Delta\mathbb{Y}$ だけでなく **LTV・リテンション・コホート** を横断した「数カ月～数年」の視点。
 - Risk & Governance Center**
 - フェアネス・制約・データ品質 を監視し、危険な施策を「Fail-Closed」で止める。

0.2 統合方針 (v1 との関係)

- 既存ナビゲーションに以下のメニューを追加 (/名称は調整可)：
 - Experiment Studio (モジュール1+2)
 - Growth Studio (モジュール3)
 - Governance (モジュール4)
- Decision Console / Portfolio / Digital Twin** には、**V2** で計算された指標を“読み取り専用”で追加表示するだけに留める。
→ これで v1 ユーザのメンタルモデルを壊さず、「高度モード」へ自然に誘導できる。

1. モジュール A - Multi-Arm Causal Design Studio

1.1 機能概要

- 目的:
treatment 列が
 - 2 クラス以上 (A/B/n)
 - 段階的 (0%, 5%, 10%, 20%)
 - 連続 (メール頻度、クーポン額)の場合でも、**因果推論と $\Delta\mathbb{Y}$ を安定して推定**する。

- **UI:** 既存 **Causal Design** の上部に「Treatment Type」セクションを追加
 - **Binary** / **Multi-Armed** / **Dose-Response** の3切り替え
 - Multi-Armed 選択時:
 - 各 treatment_arm のラベル・意味 (例: Control / Light / Medium / Aggressive) を設定できるフォーム
 - Dose-Response 選択時:
 - 処置列を「数値として解釈」+ 最低/最高 dose を指定

1.2 推定ロジック (Python)

1.2.1 多クラス処置の Potential Outcomes

treatment が 0..K の整数値の場合、

0 をコントロールとして K 個のダミーに分解し、

DR Learner / X-Learner 等をクラスごとに適用。

擬似コード:

```
# backend/engine/estimators/multi_arm_dr.py

class MultiArmDREstimator:
    def __init__(self, base_learner):
        self.base_learner = base_learner
        self.models_tau = {} # arm -> model

    def fit(self, X, T, Y):
        # T: array of 0..K (0 = control)
        arms = sorted(set(T))
        arms.remove(0)

        for a in arms:
            treated_mask = (T == a)
            control_mask = (T == 0)

            X_t = X[treated_mask]
            Y_t = Y[treated_mask]
            X_c = X[control_mask]
            Y_c = Y[control_mask]

            # DR/X-learnerなどの内部ロジックは既存バイナリ版を再利用
            model = self.base_learner()
            model.fit(X_t, Y_t, X_c, Y_c)

            self.models_tau[a] = model

    def effect(self, X):
        # 戻り値: dict arm -> tau(x)
        return {a: m.effect(X) for a, m in self.models_tau.items()}
```

1.3 UI の可視化

- **Treatment Effects Summary**
 - 各 arm ごとに $\Delta\tau$ / CAS / リスク を棒グラフ表示
- **Pairwise Comparison Matrix**
 - 各 arm のペア (a,b) について $\Delta(a \text{ vs } b)$ を表形式表示
- **Dose-Response Curve (Dose-Response モード時)**
 - $x=\text{dose}, y=E[Y|\text{dose}]$ のスムーズな曲線 (Recharts LineChart)

1.4 Wolfram での Dose-Response 検算

```
(* data: columns {"dose", "outcome", "weights"} *)
data = Import["dose_response_sample.csv"];

model = LinearModelFit[
  data[All, {"dose", "dose^2"}],
  {1, dose, dose^2},
  dose,
```

```

Weights -> data[All, "weights"]
];

(* 期待アウトカムと最適 dose を計算 *)
expectedOutcome[d_] := model[d];
bestDose = N @ argmax[expectedOutcome[d], d ∈ {0, 1, ..., 20}];

```

What this does (モジュールA)

- **ゴール:** 2値に限定されていた treatment 列を、A/B/n・段階施策・連続 dose まで拡張し、現実のキャンペーン設計（割引レベル、頻度など）をそのままモデリングできるようにする。
- **入力:** CSV 内の treatment 列が 3 クラス以上 or 数値になっているデータ。
- **出力:** 各 arm ごとの $\Delta\mathbb{E} \cdot \text{CAS} \cdot \text{リスク}$ 、arm 間の比較行列、dose-response 曲線。
- **なぜこのアプローチか:** バイナリ版の DR/X-Learner 実装をそのまま流用でき、安定性・検証コストを抑えながら多アーム対応できるため。
- **実行・検証:** Python 版 estimator のユニットテスト + Wolfram の Dose-Response notebook で、単純なシミュレーションデータに対して bias が小さいことを確認。

Expert insight

多アーム実験でも「0 vs 各 arm」を基本単位にする設計は、実務運用とデバッグを圧倒的に楽にする。
 なぜなら、「どの arm を残し、どれを削るか」の議論がすべて「Control との差」に還元でき、意思決定のフォーマットがバイナリ実験と共通化するため。

2. モジュール B - Experiment Orchestrator & Bandit Control Tower

2.1 機能概要

- **目的:** 施策配信を「一度決めて終わり」ではなく、
反応を見ながら配信比率を自動更新する仕組みを提供（メール件名テスト、LP バリエーションなど）。
- **想定ワークフロー**
 1. Experiment Studio で 候補アーム を定義
 2. Orchestrator が **KARTE / MA ツール側**に割り当てルールを配信
 3. リアルタイムイベント（send/open/click/purchase）を **cqox** にストリーム ingest
 4. バンディットアルゴリズムが **数時間〜1日ごと**に配信比率を更新

2.2 API & データフロー

- 新 API 例：
 - **POST /api/v1/experiments**
 - body: experiment_id, arms, traffic_limit, safety_constraints など
 - **GET /api/v1/experiments/{id}/allocation**
 - 現在の配信比率（各 arm の割当率）を返す
- KARTE 側からは「ユーザID, experiment_id」をキーに配信ルールを問い合わせるイメージ。

2.3 バンディットアルゴリズム (Python)

2.3.1 Thompson Sampling の骨格

```

# backend/engine/bandits/thompson.py

import numpy as np
from dataclasses import dataclass

@dataclass
class ArmState:
    alpha: float = 1.0 # 成功数 + 1
    beta: float = 1.0 # 失敗数 + 1

class BernoulliThompsonBandit:
    def __init__(self, arm_ids: list[str]):
        self.state = {arm: ArmState() for arm in arm_ids}

    def sample_allocation(self, n_samples: int = 10000) -> dict[str, float]:
        scores = {}
        for arm, s in self.state.items():
            # Beta 分布からサンプル
            score = np.random.beta(s.alpha, s.beta, size=n_samples).mean()

```

```

        scores[arm] = score
# スコアを正規化して配分比率に
total = sum(scores.values())
return {arm: val / total for arm, val in scores.items()}

def update(self, outcomes: list[tuple[str, int]]):
    """
    outcomes: list of (arm_id, reward) where reward in {0,1}
    """
    for arm_id, reward in outcomes:
        s = self.state[arm_id]
        if reward == 1:
            s.alpha += 1
        else:
            s.beta += 1

```

2.4 Orchestrator UI

- 新ページ **Experiment Studio** :
 - **Experiment List:**
 - experiment_id, status (Running / Paused / Completed) , target metric, horizon, etc.
 - **Allocation Panel:**
 - 各 arm の現在の配分比率を **棒グラフ + 数値** で表示
 - 「今すぐフリーズ」「今すぐ均等に戻す」などのボタン
 - **Online Metrics:**
 - 累積 Δ¥、conversion、探索割合 (Exploration rate) などを時間軸で表示

2.5 Wolfram でのバンディット挙動シミュレーション

```

(* 各 arm の真のコンバージョン率 *)
trueP = <|"A" -> 0.02, "B" -> 0.025, "C" -> 0.018|>;

(* Thompson Sampling をシミュレート *)
steps = 10000;
state = AssociationMap[<|"alpha" -> 1., "beta" -> 1.|> &, Keys[trueP]];

sampleAllocation[st_] := Module[{scores},
  scores = AssociationMap[
    With[{a = st[#]["alpha"], b = st[#]["beta"]},
      RandomVariate[BetaDistribution[a, b]]
    ] &,
    Keys[st]
  ];
  scores / Total[scores]
];

Do[
  alloc = sampleAllocation[state];
  arm = RandomChoice[alloc]; (* 1 ユーザ分配 *)
  reward = Boole[RandomReal[] < trueP[arm]];
  If[reward == 1, state[arm, "alpha"]++, state[arm, "beta"]++];
, {steps}];

state

```

What this does (モジュールB)

- **ゴール:** 静的なオフライン Δ¥ 評価に加えて、メール件名/バナー/LPなどを **リアルタイムで自動最適化**する Experiment Orchestrator を提供する。
- **入力:** experiment 定義 (arms, target metric)、ストリームイベント (reward=0/1 or Δ¥)。
- **出力:** 各 arm の配信比率、オンライン評価指標、実験の履歴。
- **なぜこのアプローチか:** Thompson Sampling などのベイジアンバンディットは、実装が簡潔でありながら、探索-利用バランスにおいて実務上十分な性能を持つため。
- **実行・検証:** Python + Wolfram でシミュレーションを行い、真の CTR/Δ¥ に収束することを確認。

Expert insight

オンライン最適化を入れるときの最大の落とし穴は「後から因果推論がやりにくなる」こと。

初期フェーズは **ランダム配信 + 十分なログ** を必ず確保し、その上でバンディット・RL を“2ndステージ”としてかぶせる設計が、大規模プラットフォーム (Meta / Google Ads) でも採用されているパターン。

3. モジュール C - Growth & LTV Planning Studio

3.1 機能概要

- 目的:
 - 単発キャンペーンの $\Delta\%$ ではなく、
 - 数カ月～数年スパンの **LTV** とリテンションを含めた「本当の意味での Growth」を見える化。
- 想定する入力
 - ユーザIDごとの購入履歴 (時系列)
 - キャンペーン参加フラグ (treatment)
 - コホート (初回購入月など)

3.2 数理モデル (概要)

- Survival / Retention** モデル
 - 時間 t における生存確率 $S(t)$
 - 施策あり/なしで別々に推定し、その差分から **Retention LIFT** を評価
- CLV** (顧客生涯価値)

$$CLV = \sum_{t=1}^T \mathbb{E}[R_t \cdot S(t)] \cdot \frac{1}{(1+r)^t}$$

- R_t : 期間 t の期待収益
- r : 割引率

3.3 Python 擬似コード

```
# backend/engine/ltv/ltv_estimator.py

import numpy as np
import pandas as pd

class SimpleCLVEstimator:
    def __init__(self, discount_rate: float = 0.01):
        self.discount_rate = discount_rate

    def fit(self, df_events: pd.DataFrame):
        """
        df_events: columns [user_id, t, revenue, treated]
        t: integer time period (e.g. weeks since signup)
        """
        self.df = df_events

    def _survival_curve(self, treated: bool) -> dict[int, float]:
        df = self.df[self.df["treated"] == treated]
        # 生存: その時点まで1回以上購買しているユーザ比率、など簡易定義
        surv = {}
        for t, group in df.groupby("t"):
            users_total = group["user_id"].nunique()
            active = group[group["revenue"] > 0]["user_id"].nunique()
            surv[t] = active / users_total if users_total > 0 else 0.0
        return surv

    def clv(self, treated: bool) -> float:
        surv = self._survival_curve(treated)
        clv = 0.0
        for t, s in surv.items():
            mean_rev_t = self.df[(self.df["treated"] == treated) & (self.df["t"] == t)]["revenue"].mean()
            disc = 1.0 / ((1 + self.discount_rate) ** t)
```

```

    clv += s * mean_rev_t * disc
    return float(clv)

```

3.4 Wolfram での CLV 検算

```

events = Import["customer_events.csv"]; (* user_id, t, revenue, treated *)

survivalCurve[data_, treatedVal_] := Module[{sub, groups},
  sub = Select[data, #treated == treatedVal &];
  groups = GroupBy[sub, #t &];
  AssociationMap[
    With[{g = groups[#]},
      Module[{users = Union[g[[All, "user_id"]]], active},
        active = Union[Select[g, #revenue > 0 &][[All, "user_id"]]];
        If[Length[users] == 0, 0., N[Length[active]/Length[users]]]
      ]
    ] &,
    Keys[groups]
  ];

discountRate = 0.01;

clv[data_, treatedVal_] := Module[{surv, clvVal = 0.},
  surv = survivalCurve[data, treatedVal];
  Do[
    meanRev = Mean[Select[data, #treated == treatedVal && #t == t &][[All, "revenue"]]];
    clvVal += surv[t] * meanRev / (1 + discountRate)^t;
  , {t, Keys[surv]}];
  clvVal
];

clvControl = clv[events, 0];
clvTreated = clv[events, 1];
deltaCLV = clvTreated - clvControl;

```

3.5 Growth Studio UI

- **CLV Summary:**
 - Treated/Control の CLV (¥) と差分 ΔCLV をカード表示
- **Cohort Retention Plot:**
 - 横軸: 経過週、縦軸: Retention (S(t))、複数 cohort を重ね描き
- **施策別 CLV Uplift Table:**
 - campaign_id ごとの ΔCLV, ΔRetention, コスト, ROI

What this does (モジュールC)

- **ゴール:** 施策の成否を「今月の売上」だけでなく、「数カ月後の LTV まで含めたインパクト」で評価できるようにする。
- **入力:** 施策フラグ付きの購入履歴、コホート情報。
- **出力:** CLV uplift、Retention uplift、コホート別グラフ。
- **なぜこのアプローチか:** Survival + 割引現在価値という枠組みは、金融・サブスク業界で標準化された方法であり、世界の広告プラットフォームもこの形で ROI を測っているため。
- **実行・検証:** Wolfram notebook でサンプルデータに対する CLV を計算し、Python 実装と値が一致することを確認。

Expert insight

LTV を入れるだけで「短期施策の中毒」から抜けられる。
 しかし、計算をブラックボックスにすると現場は信用しない。
 Survival × 割引現在価値のような **シンプルで説明しやすい形** を採用し、ノートブックごと顧客に見せられる状態にしておくことが、ハイエンド案件では重要になる。

4. モジュール D - Risk & Governance Center

4.1 機能概要

- 役割: 「危ない施策はそもそも実行させない」ためのゲート

- 監視するもの：
 - データ品質（リーク、極端な外れ値、サンプル数不足）
 - フェアネス / バイアス（年齢・性別などの protected 属性を跨いだ ΔY の偏り）
 - コンプライアンス制約（特定セグメントへの広告頻度制限など）

4.2 代表的なフェアネスメトリクス

たとえば男女で ΔY が大きく異なる場合、

Absolute Uplift Disparity を定義：

$$D = |\Delta Y_{\text{male}} - \Delta Y_{\text{female}}|$$

閾値 D_{max} を超えた場合、施策を「要レビュー」にする。

Python 擬似コード：

```
def uplift_disparity(df_uplift, sensitive_col: str, group_a: str, group_b: str) -> float:
    a = df_uplift[df_uplift[sensitive_col] == group_a]["delta_yen"].mean()
    b = df_uplift[df_uplift[sensitive_col] == group_b]["delta_yen"].mean()
    return abs(float(a - b))
```

4.3 Governance UI

- Quality Gates Overview**
 - 各施策ごとに「Data Quality / Fairness / Compliance」のステータス（Pass / Warn / Fail）
- Violation Detail**
 - どの属性でどの程度の偏りがあるかを棒グラフで可視化
- Export Gate との連携**
 - Fail の施策は YAML エクスポート不可にする / ワーニングを強制表示する

What this does (モジュールD)

- ゴール: ΔY だけを追って「特定属性にだけ過剰配信」「データリークで過学習」といった事故を防ぐ。
- 入力: uplift 推定結果 + 属性情報 + 企業側が設定した制約（例: 年齢差別禁止、特定カテゴリへの配信上限）
- 出力: Quality Gate の Pass/Warn/Fail、違反施策の一覧。
- 実行・検証: テスト用に「意図的にバイアスのあるデータセット」を用意し、想定通り Fail になることを pytest で検証。

Expert insight

「安全に止まる」仕組みがないと、上限単価が上がった瞬間にプロダクトは炎上リスクを抱える。
世界レベルのプラットフォームは、売上よりも先に“どの条件なら絶対に配信しないか”をコード化する。
Governance Center はまさにその「コード化された倫理」を UI として露出させる役割になる。

最後に：V2 を既存可視化にどうつなぐか

- Decision Console v2**
 - サマリーカードに「Online Experiments Running」「6CLV」など V2 指標を追加するが、操作はできない（読み取り専用）。
- Portfolio v2**
 - ポートフォリオの各 policy に「LTV uplift」「Fairness risk 指標」を列として追加。
- Digital Twin v2**
 - Growth Studio で推定した CLV uplift を Twin のペルソナにも反映し、「この施策を打つと 1年後 LTV がいくら伸びるか」をシミュレート。

全体 What this does (V2 全体)

- ゴール:
CQOx を「因果推論ダッシュボード」から
“実験設計～オンライン最適化～長期成長～ガバナンス”までを一気通貫で扱う世界最高水準のマーケティング OS に引き上げる。
- 入力:
既存の CSV / イベントログに加え、experiment 定義、LTV用の購入履歴、制約条件。
- 出力:
 - Multi-Arm 対応の因果デザイン
 - オンラインバンディットによる自動配信比率

- LTV / Retention を含んだ Growth 評価
- フェアネス・安全装置付きの Governance レイヤ
- **なぜこの構成か:**
 - 「ページを増やしても、意思決定の単位が分かれていればユーザは迷わない」
 - 逆に、全てを Decision Console に押し込むと、世界レベルどころか **普通のBIツール以下** になるリスクが高い。
そこで、
 - Off-line (Causal & Portfolio)
 - On-line (Experiment Orchestrator)
 - Long-term (Growth Studio)
 - Safety (Governance)

の4レイヤに明確に分離した。
- **実行・検証:**
各モジュールについて Python 実装 + Wolfram notebook を用意し、
シミュレーションデータでアルゴリズムの挙動を検算することを前提としている。

Expert insight (Google/Meta/NASA レベル)

「一枚のダッシュボード」で完結させようとする、必ず重要な何かを諦めることになる。

世界トップの広告/成長チームは、

- *Offline* 因果推論
- *Online* バンディット / RL
- LTV / コホート
- フェアネス & ガバナンス

をレイヤと責任範囲で切り分けたうえで、それぞれに最適な **UI** と指標を設計している。

CQOx V2 も同じ構造を採用することで、「これ以上削ると世界水準から落ちる」というラインまで機能を押し上げられる。